AI

Task for low attendance students

Name: Arjun Mankuskar

PRN:202401110038

Branch: CSE (AIML)

Batch: A2

CODE:-

assigment.py

```python
import yfinance as yf
import pandas as pd

# Settings
ticker_symbol = 'MSFT'
start = '2023-11-01'
end = '2025-11-01'

#  1. Fetch Data
data = yf.download(ticker_symbol, start=start, end=end)

# 2. Date-Time Features
# We need 'Date' as a column to work with it
data.reset_index(inplace=True)
data['Date'] = pd.to_datetime(data['Date'])

# Extract features from the 'Date' column
data['day_of_month'] = data['Date'].dt.day
data['month'] = data['Date'].dt.month
data['weekday'] = data['Date'].dt.weekday  # Monday=0, Sunday=6

# 3. Lag Features
# 'shift(1)' gets the value from 1 row before (yesterday)
data['lag_1_day'] = data['Close'].shift(1)
# 'shift(7)' gets the value from 7 rows before (last week)
data['lag_7_days'] = data['Close'].shift(7)

#  4. Rolling Mean Features
# 'rolling(window=7)' looks at the last 7 days and gets the average
data['rolling_mean_7_day'] = data['Close'].rolling(window=7).mean()
# Same thing, but for a 30-day window
data['rolling_mean_30_day'] = data['Close'].rolling(window=30).mean()

#  5. Clean and Show Output
# Drop rows with empty (NaN) values created by shift() and rolling()
final_data = data.dropna()

# Set 'Date' back as the index for a clean table
final_data.set_index('Date', inplace=True)
```

Name: Arjun Mankuskar (PRN:202401110038)

```python
# Define which columns we want to see in the final output
columns_to_show = [
    'Close',
    'day_of_month',
    'month',
    'weekday',
    'lag_1_day',
    'lag_7_days',
    'rolling_mean_7_day',
    'rolling_mean_30_day'
]

# Print the first 5 rows
print("\n--- Sample Output (Head) ---")
print(final_data[columns_to_show].head(10))

# Print the last 5 rows
print("\n--- Sample Output (Tail) ---")
print(final_data[columns_to_show].tail(10))
```

OUTPUT:-

```
PS C:\Users\Arjun\OneDrive\Desktop\Collage\AI> python -u "c:\Users\Arjun\OneDrive\Desktop\Collage\AI\assigment.py"
c:\Users\Arjun\OneDrive\Desktop\Collage\AI\assigment.py:10: FutureWarning: YF.download() has changed argument auto_adjust default to True
  data = yf.download(ticker_symbol, start=start, end=end)
[**********100%**********]  1 of 1 completed

--- Sample Output (Head) ---
Price            Close day_of_month month weekday   lag_1_day  lag_7_days rolling_mean_7_day rolling_mean_30_day
Ticker            MSFT
Date
2023-12-13  369.560577           13    12       2  369.570435  364.397766         367.580641          364.530342
2023-12-14  361.229004           14    12       3  369.560577  367.734344         366.651306          365.206902
2023-12-15  365.967377           15    12       4  361.229004  364.062134         366.923484          365.967523
2023-12-18  367.862701           18    12       0  365.967377  366.184570         367.163217          366.644205
2023-12-19  368.464844           19    12       1  367.862701  369.422394         367.026424          367.218470
2023-12-20  365.858734           20    12       2  368.464844  366.530029         366.930525          367.574510
2023-12-21  368.741241           21    12       3  365.858734  369.570435         366.812068          367.938955
2023-12-22  369.767883           22    12       4  368.741241  369.560577         366.841684          368.420048
2023-12-26  369.846863           26    12       1  369.767883  361.229004         368.072806          368.608883
2023-12-27  369.264435           27    12       2  369.846863  365.967377         368.543815          368.876491

--- Sample Output (Tail) ---
Price            Close day_of_month month weekday   lag_1_day  lag_7_days rolling_mean_7_day rolling_mean_30_day
Ticker            MSFT
Date
2025-10-20  516.789978           20    10       0  513.580017  522.400024         513.427137          513.398331
2025-10-21  517.659973           21    10       1  516.789978  510.959991         514.384277          514.039996
2025-10-22  520.539978           22    10       2  517.659973  514.049988         515.311419          514.712329
2025-10-23  520.559998           23    10       3  520.539978  513.570007         516.309989          515.363995
2025-10-24  523.609985           24    10       4  520.559998  513.429993         517.764274          515.820995
2025-10-27  531.520020           27    10       0  523.609985  511.609985         520.608564          516.359663
2025-10-28  542.070007           28    10       1  531.520020  513.580017         524.678563          517.460663
2025-10-29  541.549988           29    10       2  542.070007  516.789978         528.215707          518.511663
2025-10-30  525.760010           30    10       3  541.549988  517.659973         529.372855          519.088663
2025-10-31  517.809998           31    10       4  525.760010  520.539978         528.982858          519.084663
PS C:\Users\Arjun\OneDrive\Desktop\Collage\AI>
```

Name: Arjun Mankuskar (PRN:202401110038)

How These Features Help Find Trends

To a machine learning model, a time-series is just a list of numbers. It doesn't know what "yesterday" was or that "December" comes every year. These new features fix this by giving the model crucial **context**. We are essentially providing it with a **calendar**, a **memory**, and a way to see the **"big picture" trend**.

1. Date-Time Features (The "Calendar")

- **What they do:** These features break the date down into its components: day_of_month, month, and weekday.
- **How they help:** This allows the model to spot **seasonality**, or patterns that happen at regular times. For example, the model can learn if there's a "day-of-week effect" (like prices dropping on Mondays) or a "month effect" (like a holiday sales spike in November).
- **Impact:** The model learns to distinguish between a predictable, seasonal dip and the start of a *new, negative trend*. It learns not to "panic" just because it's a "slow August."

2. Lag Features (The "Memory")

- **What they do:** These features are the model's short-term memory. They show it exactly what the price was 1 day ago (lag_1_day) and 7 days ago (lag_7_days).
- **How they help:** This is all about capturing **momentum**. The model can directly compare the current price to the price yesterday or last week.
- **Impact:** The model can learn rules from this comparison. If the current price is higher than both its 1-day and 7-day lags, it's a strong signal of an upward trend. It helps the model see the difference between a single "good day" and a real "winning streak."

3. Rolling Mean (The "Smooth Trend Line")

- **What they do:** A daily stock chart is "noisy" with lots of spiky up-and-down movements. A rolling mean **smooths out this noise** by averaging the price over a set window (e.g., the last 7 or 30 days).
- **How they help:** They create a clean, smooth trend line for the model to follow. The rolling_mean_7_day shows the short-term trend, while the rolling_mean_30_day shows the "big picture" long-term trend.
- **Impact:** The model can now compare the current price to these averages. If the price is *above* the 30-day average, it's a clear sign of a long-term uptrend. Even better, if the *short-term 7-day average* crosses *above* the *long-term 30-day average*, it's a powerful signal that positive momentum is accelerating.

Name: Arjun Mankuskar (PRN:202401110038)